

Student Claw — Architectural Blueprint & System Design Specification

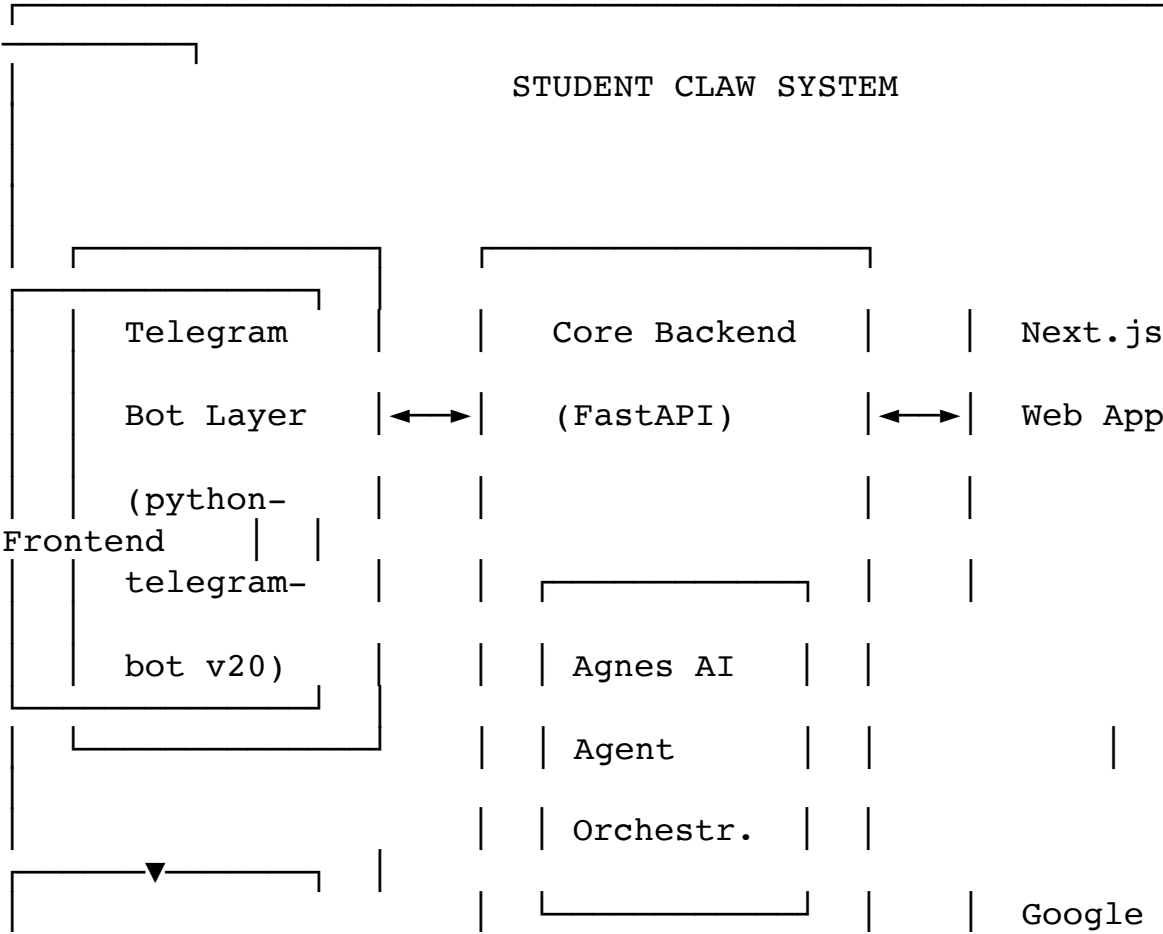
Version: 1.0 | **Classification:** Internal Engineering Design Document | **Date:** 2026-06-25

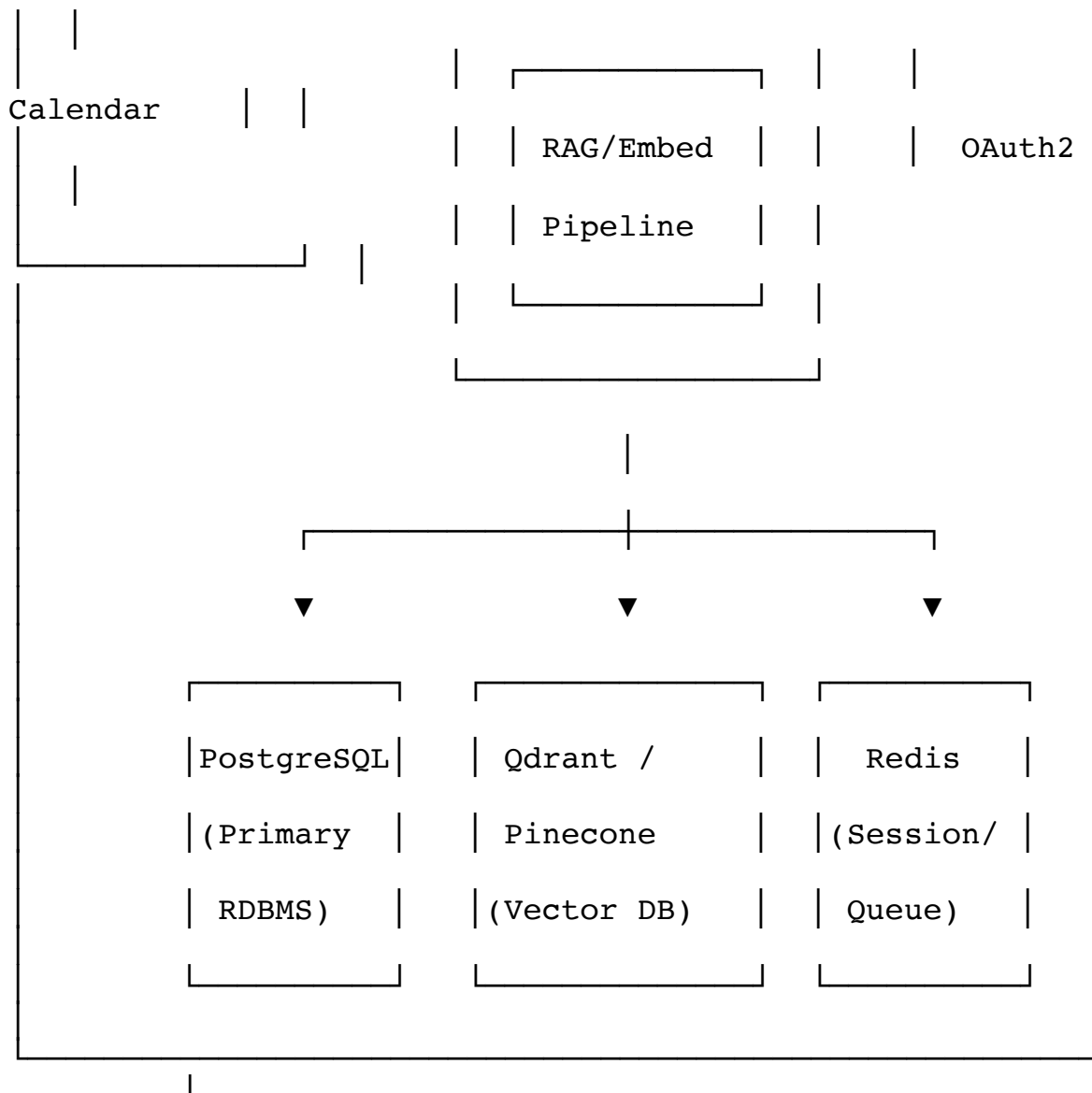
Table of Contents

- 1 [System Architecture Overview](#)
- 2 [Data Modeling & Database Schema](#)
- 3 [Agnes AI Agent Execution & Tool Calling](#)
- 4 [Cross-Platform Onboarding & State Synchronization](#)
- 5 [RAG & Cache Cleansing Pipeline](#)
- 6 [Frontend UI/UX Technical Strategy](#)
- 7 [Infrastructure & Deployment Topology](#)
- 8 [Security Architecture](#)

1. System Architecture Overview

1.1 High-Level Component Map





1.2 Technology Stack Decisions

Layer	Technology	Rationale
Bot Runtime	Python-telegram-bot v20 (async)	Native async, webhook support, robust handler system
API Backend	FastAPI (Python)	Co-language with bot, async-native, OpenAPI auto-docs
Frontend	Next.js 14 (App Router)	SSR for auth pages, RSC for dashboard, edge-ready
Primary DB	PostgreSQL 16	ACID compliance, JSONB for flexible metadata, row-level security
Vector DB	Qdrant (self-hosted)	Native metadata filtering, no per-vector pricing, GDPR-friendly
Cache / Queue	Redis 7	Job queuing for async pipelines, session caching, pub/sub for real-time sync

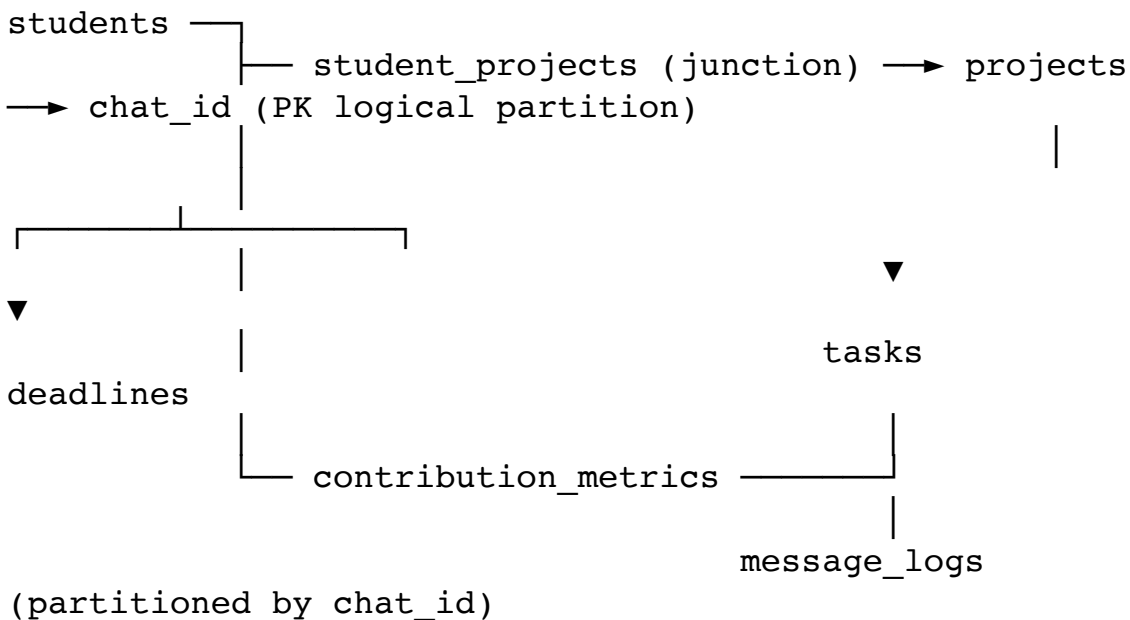
Embeddings	Agnes AI / embeddings endpoint	Unified vendor, avoids dual-API key management
File Storage	MinIO (S3-compatible)	Self-hostable, bucket isolation per chat_id
OCR	Tesseract via pytesseract	Open-source, no external API needed for image text extraction
PDF/PPTX Parse	PyMuPDF + python-pptx	Best-in-class extraction fidelity

2. Data Modeling & Database Schema

2.1 Design Principles

- **Absolute Tenant Isolation:** Every AI-facing query is scoped to a `chat_id`. No cross-project data leakage is architecturally possible.
- **Soft Deletes:** No hard `DELETE` on core entities. A `deleted_at` timestamp and `is_active` flag on critical tables enables audit trails and safe cache-cleansing rollback windows.
- **Normalized + JSONB Hybrid:** Strict normalization for relational integrity, JSONB for flexible AI-generated metadata (contribution reasons, skill tags) that evolves without schema migrations.

2.2 Entity-Relationship Overview



2.3 Full PostgreSQL Schema

Table: **students**

Column	Type	Constraints	Notes
--------	------	-------------	-------

id	UUID	PK, DEFAULT gen_random_uuid()	Internal system ID
display_name	VARCHAR(100)	NOT NULL	Full display name
username	VARCHAR(50)	UNIQUE, NOT NULL	Web login handle
password_hash	TEXT	NOT NULL	bcrypt hash, never store plaintext
telegram_username	VARCHAR(50)	UNIQUE, NULLABLE	Without @, linked after Telegram verification
telegram_user_id	BIGINT	UNIQUE, NULLABLE	Telegram's internal numeric ID, set during /link flow
email	VARCHAR(255)	UNIQUE, NULLABLE	Optional for notifications
google_calendar_refresh_token	TEXT	NULLABLE	Encrypted at rest, OAuth2 offline token
google_calendar_linked_at	TIMESTAMPZ	NULLABLE	—
created_at	TIMESTAMPZ	DEFAULT NOW()	—
updated_at	TIMESTAMPZ	DEFAULT NOW()	Trigger-maintained
is_active	BOOLEAN	DEFAULT TRUE	Soft account disable

Table: projects

Column	Type	Constraints	Notes
id	UUID	PK, DEFAULT gen_random_uuid()	Internal project ID
project_key	VARCHAR(64)	UNIQUE, NOT NULL	Public-facing linkage key. Derived from chat_id via HMAC-SHA256 (first 16 hex chars). Never exposes raw chat_id.
chat_id	BIGINT	NOT NULL, UNIQUE	Telegram group's canonical identifier

name	VARCHAR(200)	NOT NULL	Extracted from group title or manually set
module_code	VARCHAR(50)	NULLABLE	e.g., "50.012", "10.009"
status	ENUM('active', 'archived', 'cleared')	DEFAULT 'active'	Drives cache-cleanse eligibility
vector_name_space	VARCHAR(100)	NOT NULL, UNIQUE	Maps 1:1 to Qdrant collection name. Format: project_{chat_id}
qdrant_point_count	INTEGER	DEFAULT 0	Denormalized counter for dashboard display
created_at	TIMESTAMPTZ	DEFAULT NOW()	—
cleared_at	TIMESTAMPTZ	NULLABLE	Populated on cache cleanse

Table: student_projects (Junction / Many-to-Many)

Column	Type	Constraints	Notes
id	UUID	PK	—
student_id	UUID	FK → students.id, NOT NULL	—
project_id	UUID	FK → projects.id, NOT NULL	—
role	ENUM('member', 'lead', 'observer')	DEFAULT 'member'	AI can escalate role
joined_at	TIMESTAMPTZ	DEFAULT NOW()	—
linked_via	ENUM('telegram', 'web_key')	NOT NULL	How the linkage was established
—	—	UNIQUE(student_id, project_id)	Prevents duplicate memberships

Table: tasks

Column	Type	Constraints	Notes
id	UUID	PK	—

project_id	UUID	FK → projects.id, — NOT NULL
assigned_to_student_id	UUID	FK → students.id, Null = unassigned, NOT NULLABLE
title	VARCHAR(500)	NOT NULL —
description	TEXT	NOT NULLABLE —
source	ENUM('ai_delegated', 'manual', 'deadline_extracted')	NOT NULL Provenance tracking, DEFAULT
status	ENUM('pending', 'in_progress', 'done', 'dropped')	NOT NULL 'pending' —
priority	SMALLINT	DEFAULT 1=High, 2=Medium, 3=Low, T 2
google_calendar_event_id	TEXT	NOT NULLABLE For bidirectional sync, DEFAULT
created_at	TIMESTAMPTZ	NOT NULL NOW() —
updated_at	TIMESTAMPTZ	NOT NULL NOW() —
deleted_at	TIMESTAMPTZ	NOT NULLABLE Soft delete, DEFAULT
metadata	JSONB	AI-generated context: {"confidence": 0.92, "source_message_id": 12345}, DEFAULT T '{}'

Table: deadlines

Column	Type	Constraints	Notes
id	UUID	PK	—
project_id	UUID	FK → projects.id, NOT NULL	—
task_id	UUID	FK → tasks.id, NULLABLE	Optional linkage to a task
title	VARCHAR(500)	NOT NULL	—
due_date	TIMESTAMPTZ	NOT NULL	—
extracted_by	ENUM('ai', 'manual')	NOT NULL	—
source_message_log_id	UUID	FK → message_logs.id, NULLABLE	Provenance for AI extractions
is_confirmed	BOOLEAN	DEFAULT FALSE	User must confirm AI-extracted deadlines
google_calendar_event_id	TEXT	NULLABLE	—
created_at	TIMESTAMPTZ	DEFAULT NOW()	—

Table: message_logs

Design Note: This table uses PostgreSQL **declarative range partitioning** on `chat_id` (LIST partitioning by hashed bucket) to ensure that full table scans for a given project never touch other projects' rows. For smaller deployments, a `chat_id` index suffices.

Column	Type	Constraints	Notes
id	UUID	PK	—
chat_id	BIGINT	NOT NULL, INDEX	Partition key
project_id	UUID	FK → projects.id, NOT NULL	—
telegram_message_id	BIGINT	NOT NULL	Telegram's own message ID

sender_telegram_username	VARCHAR(50)	NULLABLE	—
sender_telegram_user_id	BIGINT	NULLABLE	—
content_type	ENUM('text','image','document','voice')	NOT NULL	—
raw_text	TEXT	NULLABLE	For text messages
extracted_text	TEXT	NULLABLE	OCR/parse output for images/docs
file_storage_path	TEXT	NULLABLE	MinIO path: s3://chat-{chat_id}/docs/{filename}
file_mime_type	VARCHAR(100)	NULLABLE	—
is_vectorized	BOOLEAN	DEFAULT FALSE	Tracks embedding pipeline completion
qdrant_point_ids	UUID[]	DEFAULT ' {} '	Qdrant point IDs for traceability and deletion
received_at	TIMESTAMPTZ	NOT NULL	Telegram message timestamp
created_at	TIMESTAMPTZ	DEFAULT NOW()	—
deleted_at	TIMESTAMPTZ	NULLABLE	Soft delete for cache cleanse

Table: contribution_metrics

Column	Type	Constraints	Notes
id	UUID	PK	—
project_id	UUID	FK → projects.id, NOT NULL	—
student_id	UUID	FK → students.id, NOT NULL	—

scored_at	TIMEST AMPTZ	DEFAULT NOW()	—
score_value	NUMERI C(4,2)	NOT NULL, CHECK(score_va lue BETWEEN 0 AND 10)	0.00–10.00
score_reaso n	TEXT	NOT NULL	AI natural language justification
scoring_win dow_start	TIMEST AMPTZ	NOT NULL	What time range was evaluated
scoring_win dow_end	TIMEST AMPTZ	NOT NULL	—
metadata	JSONB	DEFAULT '{}'	Breakdown: {"messages_sent": 42, "tasks_completed": 3, "files_uploaded": 1}

Table: project_link_tokens

Used during the secure onboarding linkage flow. Short-lived verification tokens.

Column	Type	Constraints	Notes
id	UUID	PK	—
token	VARCHA R(64)	UNIQUE, NOT NULL	Cryptographically random 32- byte hex
chat_id	BIGINT	NOT NULL	The Telegram group this token is for
project_id	UUID	FK → projects.id, NOT NULL	—
created_at	TIMEST AMPTZ	DEFAULT NOW()	—
expires_at	TIMEST AMPTZ	NOT NULL	created_at + INTERVAL '15 minutes'
consumed_at	TIMEST AMPTZ	NULLABLE	Set on use; single-use enforcement
consumed_by_s tudent_id	UUID	FK → students.id, NULLABLE	—

2.4 Vector Database Strategy (Qdrant)

Collection Structure:

One Qdrant collection per project, named `project_{chat_id}`. This provides hard namespace isolation — a misconfigured query cannot accidentally retrieve another project's vectors because collections are entirely separate indexes.

Point Schema per Vector:

```
Point {
  id:          UUID          # Mirrors
message_logs.qdrant_point_ids[]
  vector:      float[1536]    # Agnes AI embedding
dimensions
  payload: {
    chat_id:      BIGINT,
    project_id:    STRING (UUID),
    message_log_id: STRING (UUID),
    content_type:  STRING,
    sender_username: STRING,
    received_at:   ISO8601 STRING,
    chunk_index:   INTEGER,    # For multi-chunk
documents
    source_filename: STRING,    # Null for plain text
    text_snippet:    STRING     # First 200 chars for
result display
  }
}
```

Why per-collection rather than single-collection + metadata filter?

- Deletion at cache-cleanse time is a single `DELETE /collections/{name}` API call — atomic, instant, no orphan vectors.
- Avoids the performance degradation of filtering a global collection with millions of points.
- Aligns with Qdrant's recommended multi-tenancy pattern for high-isolation requirements.

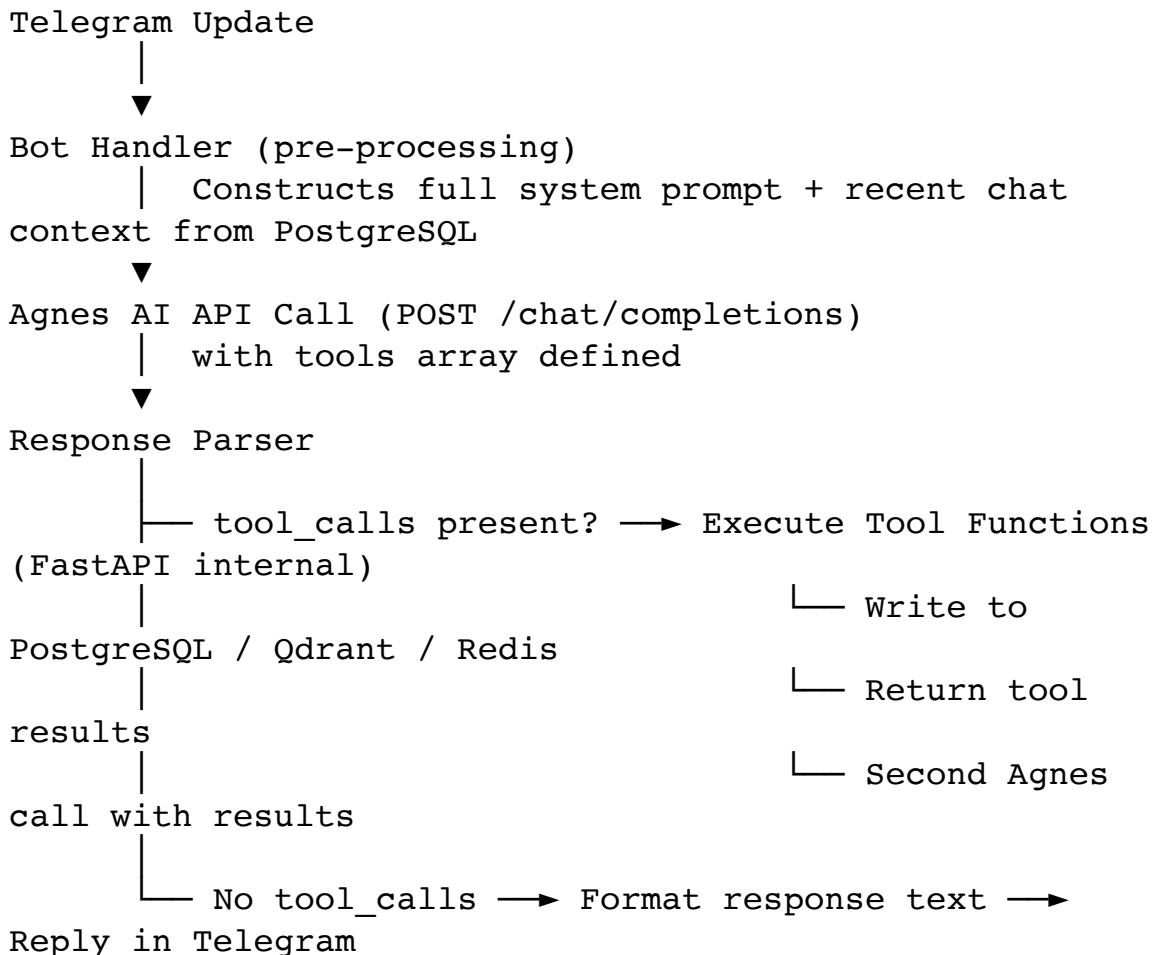
Chunking Strategy:

Content Type	Chunk Size	Overlap	Strategy
Chat messages	Sliding window of 10 messages	2 messages	Conversational context preservation
PDF documents	512 tokens	64 tokens	Recursive character splitter
PPTX slides	Per-slide	0	Slides are semantically atomic
Image OCR text	256 tokens	32 tokens	OCR output tends to be fragmented

3. Agnes AI Agent Execution & Tool Calling

3.1 Agent Architecture Pattern

Agnes AI is consumed as a stateless LLM API. The **orchestration logic lives in the FastAPI backend**, not inside Agnes. Agnes executes tool calls within a single request-response cycle; the backend manages state persistence across turns.



3.2 System Prompt Architecture

The system prompt sent to Agnes on every relevant message has five sections injected dynamically:

- 1 **Role Definition:** Agnes's persona as a project management AI assistant for students
- 2 **Project Context Block:** Project name, module code, member list (with Telegram usernames)
- 3 **Recent Message Window:** Last N messages from `message_logs` for the `chat_id`
- 4 **Available Tools:** The full JSON schema array (defined below)
- 5 **Behavioral Constraints:** Never invent deadlines not explicitly mentioned, always attribute tasks to real members only

3.3 Tool Calling JSON Schemas

All tools follow the OpenAI function-calling specification, which Agnes AI's compatible endpoint accepts.

Tool: upsert_deadline

```
{
  "type": "function",
  "function": {
    "name": "upsert_deadline",
    "description": "Extract and store a project deadline when a student explicitly mentions a due date or submission date in the conversation. Only call this when a date is unambiguously stated. Upsert behavior: if a deadline with the same title already exists for this chat_id, update its due_date.",
    "parameters": {
      "type": "object",
      "properties": {
        "chat_id": {
          "type": "integer",
          "description": "The Telegram group chat ID. This is always provided by the system context and must not be inferred from conversation."
        },
        "task_title": {
          "type": "string",
          "description": "A concise, human-readable title for the deadline. Example: 'Final Report Submission', 'Prototype Demo Video'. Maximum 200 characters.",
          "maxLength": 200
        },
        "due_date": {
          "type": "string",
          "format": "date-time",
          "description": "The deadline in ISO 8601 format with timezone. If only a date is mentioned (no time), default to 23:59:00 in Singapore Time (UTC+8). Example: '2026-07-15T23:59:00+08:00'."
        },
        "source_message_id": {
          "type": "integer",
          "description": "The Telegram message_id of the message that contained the deadline mention. Used for provenance tracking."
        }
      }
    }
  }
}
```

```

        "confidence": {
            "type": "number",
            "minimum": 0.0,
            "maximum": 1.0,
            "description": "Your confidence that this is a
genuine, explicitly stated deadline (not speculative).
Values below 0.7 should not trigger this tool."
        }
    },
    "required": ["chat_id", "task_title", "due_date",
"source_message_id", "confidence"],
    "additionalProperties": false
}
}
}

```

Tool: delegate_task

```

{
    "type": "function",
    "function": {
        "name": "delegate_task",
        "description": "Create and assign a task to a
specific group member. Only call this when: (a) a member
explicitly volunteers ('I'll handle X'), (b) a member is
explicitly assigned by another member, or (c) a member's
demonstrated expertise makes them the unambiguous choice
AND delegation was requested. Never assign tasks
speculatively.",
        "parameters": {
            "type": "object",
            "properties": {
                "chat_id": {
                    "type": "integer",
                    "description": "The Telegram group chat ID from
system context."
                },
                "telegram_username": {
                    "type": "string",
                    "description": "The Telegram username (without
@) of the member receiving the task. Must be a member
listed in the project context. Example: 'john_doe'."
                },
                "task_description": {
                    "type": "string",

```

```

        "description": "A detailed, actionable
description of the task. Should include what needs to be
done, any relevant context from the conversation, and
measurable completion criteria if discernible.",
        "maxLength": 1000
    },
    "task_title": {
        "type": "string",
        "description": "Short title for the task card.
Maximum 200 characters.",
        "maxLength": 200
    },
    "priority": {
        "type": "integer",
        "enum": [1, 2, 3],
        "description": "Task priority: 1=High (has a
deadline within 48h or explicitly urgent), 2=Medium
(default), 3=Low."
    },
    "related_deadline_title": {
        "type": "string",
        "description": "Optional: the task_title of an
existing deadline this task contributes to. Used to link
tasks to deadlines in the UI.",
        "nullable": true
    },
    "delegation_rationale": {
        "type": "string",
        "description": "Brief explanation of why this
person was selected. Example: 'User volunteered in
message 4521' or 'Member indicated expertise in machine
learning'. This is stored for transparency."
    }
},
    "required": ["chat_id", "telegram_username",
"task_description", "task_title", "priority",
"delegation_rationale"],
    "additionalProperties": false
}
}
}

```

Tool: log_contribution_metric

```
{
```

```

"type": "function",
"function": {
  "name": "log_contribution_metric",
  "description": "Score a member's contribution to the
project over a specified evaluation window. Only call
this when explicitly requested by a group member (e.g.,
'/rate @username' or '/contribution report'). Never call
this autonomously. Scoring must be evidence-based from
conversation history.",
  "parameters": {
    "type": "object",
    "properties": {
      "chat_id": {
        "type": "integer",
        "description": "The Telegram group chat ID from
system context."
      },
      "telegram_username": {
        "type": "string",
        "description": "The Telegram username (without
@) being evaluated."
      },
      "score_value": {
        "type": "number",
        "minimum": 0.0,
        "maximum": 10.0,
        "description": "Numeric contribution score from
0.00 to 10.00. Use one decimal place. Calibrate against
the full group: a perfectly equal distribution would give
all members a 5.0."
      },
      "score_reason": {
        "type": "string",
        "description": "A detailed, objective,
evidence-based justification. Must cite specific actions:
tasks completed, files submitted, decisions made, help
provided. Avoid subjective language. Example: 'Completed
4/5 assigned tasks, uploaded the system architecture
diagram, clarified requirements in 12 messages. Did not
contribute to the testing phase.'",
        "maxLength": 2000
      },
      "scoring_window_start": {
        "type": "string",

```

```

        "format": "date-time",
        "description": "Start of the evaluation period
in ISO 8601."
    },
    "scoring_window_end": {
        "type": "string",
        "format": "date-time",
        "description": "End of the evaluation period in
ISO 8601. Defaults to now."
    },
    "evidence_message_ids": {
        "type": "array",
        "items": { "type": "integer" },
        "description": "Array of Telegram message_ids
that were used as evidence for this score. Used for audit
trail display in the web dashboard.",
        "maxItems": 50
    }
},
"required": ["chat_id", "telegram_username",
"score_value", "score_reason", "scoring_window_start",
"scoring_window_end"],
"additionalProperties": false
}
}
}

```

Tool: search_project_context

This tool enables Agnes to perform RAG retrieval mid-generation rather than relying solely on the pre-injected context window.

```

{
  "type": "function",
  "function": {
    "name": "search_project_context",
    "description": "Semantically search the project's
stored conversation history, documents, and files to
answer specific questions. Use this before answering any
question that requires recalling specific past
information not present in the current context window.",
    "parameters": {
      "type": "object",
      "properties": {
        "chat_id": {
          "type": "integer",

```



```

        "description": "Scopes the search to this
project only. Never search across projects."
    },
    "query": {
        "type": "string",
        "description": "The semantic search query.
Rephrase the user's question as a search query optimized
for semantic similarity. Example: for 'what did we decide
about the database?', use 'database technology decision
architecture choice'.",
        "maxLength": 500
    },
    "content_type_filter": {
        "type": "string",
        "enum": ["all", "text", "image", "document"],
        "description": "Optionally restrict results to
a specific content type. Default: 'all'.",
        "default": "all"
    },
    "top_k": {
        "type": "integer",
        "minimum": 1,
        "maximum": 20,
        "description": "Number of results to retrieve.
Default 8.",
        "default": 8
    }
},
"required": ["chat_id", "query"],
"additionalProperties": false
}
}
}

```

3.4 Agentic Loop Control

Agnes AI may require multiple tool calls in a single turn (e.g., search context → extract deadline → delegate task). The backend implements a **bounded agentic loop**:

- **Max iterations:** 5 tool call rounds per user message
- **Loop termination:** When Agnes returns a `finish_reason` of "stop" (no more tool calls)
- **Timeout:** 30 seconds total per agentic chain
- **Circular call detection:** Backend tracks called tool signatures in a set per loop; duplicate calls with identical arguments within the same loop are dropped

4. Cross-Platform Onboarding & State Synchronization

4.1 Complete Onboarding Sequence Diagram

ACTOR: Student (Web Browser)

ACTOR: Student (Telegram)

SYSTEM: FastAPI Backend

SYSTEM: PostgreSQL

SYSTEM: Bot

PHASE 1: WEB ACCOUNT CREATION

Student(Web) —[POST /api/auth/
register]—> Backend
 {name, username, password,
 telegram_username}

|

┌───────────▼───────────┐		
Hash password		
(bcrypt, cost=12)		
INSERT students		
(telegram_user_id		
NULL for now)		=

└───────────┘

|

Backend —[201 Created + JWT (15min) + refresh_token]—> Student(Web)

PHASE 2: BOT ADDITION & PROJECT REGISTRATION

Student(Telegram) —[Adds @StudentClawBot to group
chat]→ Telegram

|
Telegram —[my_chat_member update event]————→
Bot

|

┌───────────▼───────────┐

Check: project for |
this chat_id |
exists? NO |

|

INSERT projects |
chat_id = X |
project_key = |
HMAC(X, SECRET) |
[:16] |
status = active |

|

CREATE Qdrant |
collection |
project_{chat_id} |

└───────────┘

|

Bot —[Sends welcome message to group]————→
Telegram Group
 "✓ Student Claw activated!
 Project Key: SC-4f8a2b9e1c3d7f0a
 Share this key in the web dashboard to link your
account."

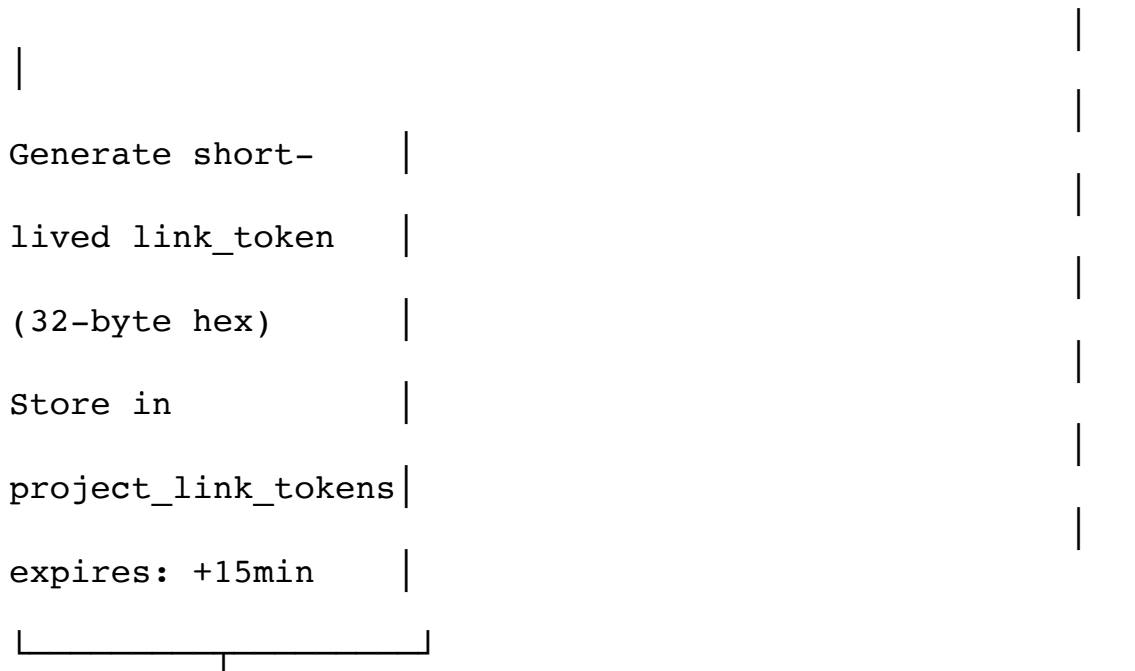
PHASE 3: WEB DASHBOARD PROJECT LINKING

Student(Web) —[POST /api/projects/
link]————→ Backend
 {project_key: "SC-4f8a2b9e1c3d7f0a"}
 [Bearer JWT in Authorization header]

|

┌──────────▼──────────┐

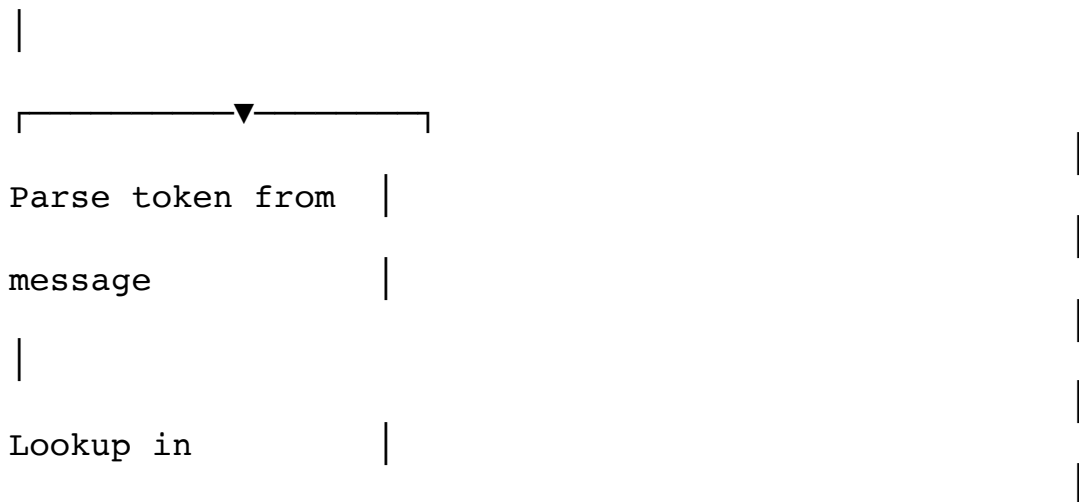
Validate JWT		
Lookup project_key		
projects table		in
student linked		Has
telegram_username?		
→ Check if		YES
that username is		
the Telegram		in
group (via stored		
member list)		



Backend —[200 OK, {verification_required: true,}]—> Student(Web)
instructions: "Send /verify {token} in your Telegram group within 15 minutes"

PHASE 4: TELEGRAM IDENTITY VERIFICATION

Student(Telegram) —[Sends "/verify SC-TOKEN-XYZ" in group]—> Bot



project_link_tokens		
Validate:		
Not expired		-
Not consumed		-
chat_id matches		-
current group		
UPDATE students		
telegram_user_id		
message.from.id		=
INSERT		
student_projects		
(student, project		
linked_via=		
'telegram')		
MARK token		
consumed_at=NOW()		
PUBLISH to Redis		

```

channel:      |
              |
"project_updates |
              | :
{project_id}"   |
              |
event: member_joined|
|_____|
|

```

```

Bot —[Private message to
student]————→ Telegram
      "✓ Verified! Your account is now linked to [Project
Name]."

```

PHASE 5: REAL-TIME SYNC TO WEB DASHBOARD

```

Redis Pub/Sub —[Event:
member_joined]———— Backend SSE

```

```

Backend maintains
connection per
authenticated student

```

```

Backend —[SSE: {type:"member_joined",
project_id:...}]→ Student(Web)

```

```

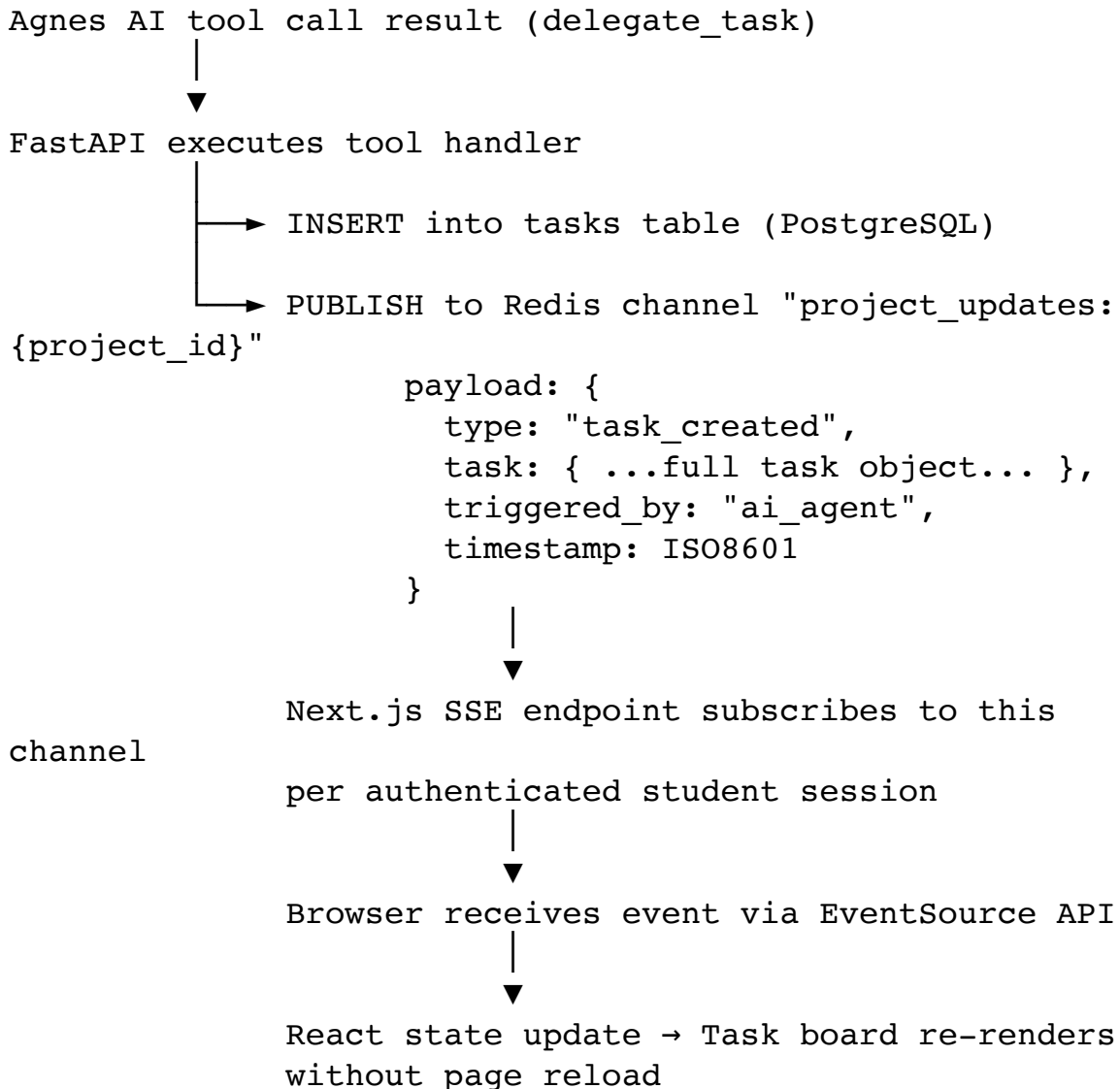
Next.js client receives SSE event —→ React state update
                                   —→ Dashboard re-
renders with
                                   new project
visible

```

4.2 Real-Time Synchronization Architecture

Mechanism: Server-Sent Events (SSE) over the Next.js API routes, backed by Redis Pub/Sub.

Flow for AI-generated changes (e.g., task delegation):



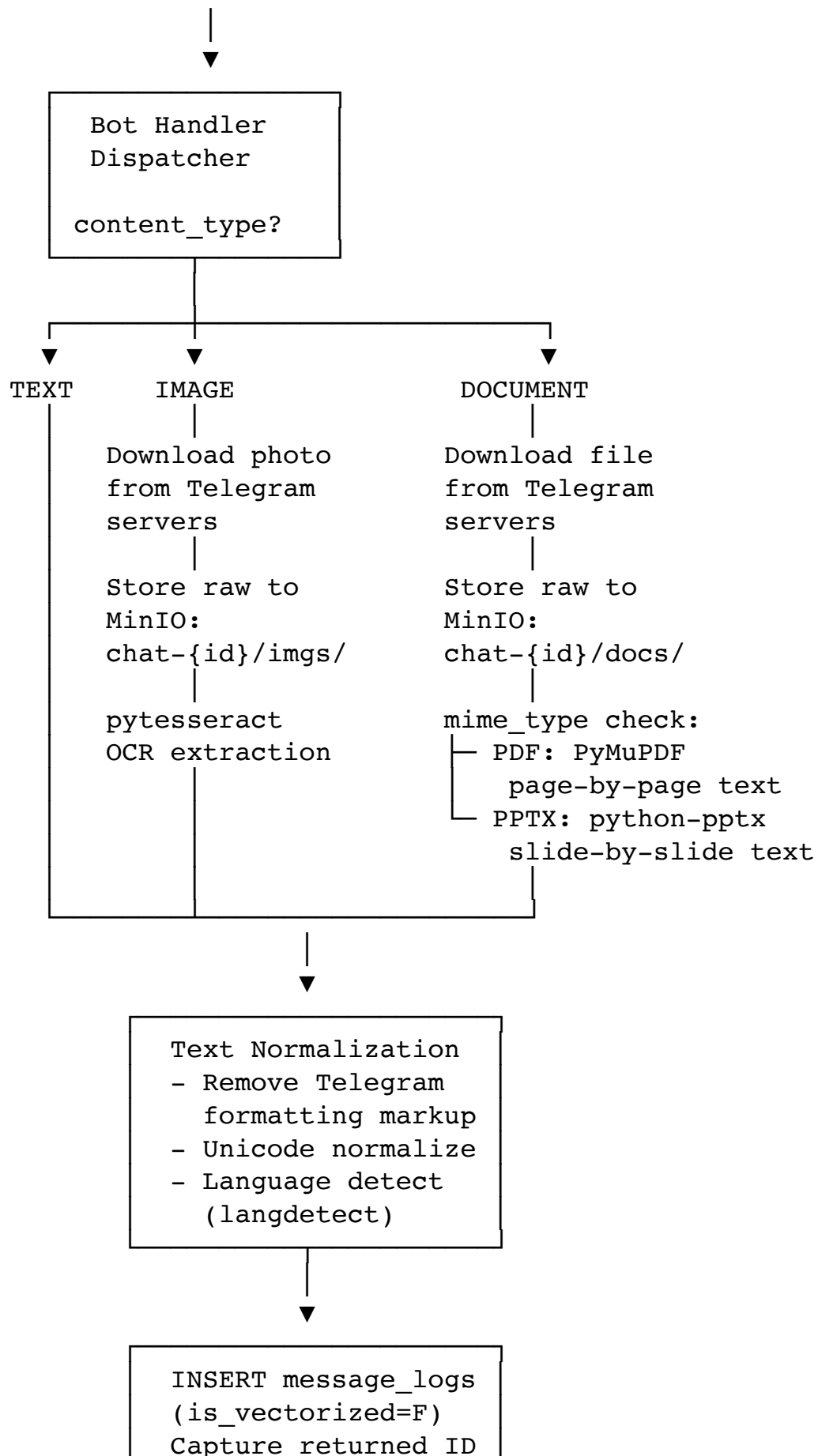
SSE Connection Management:

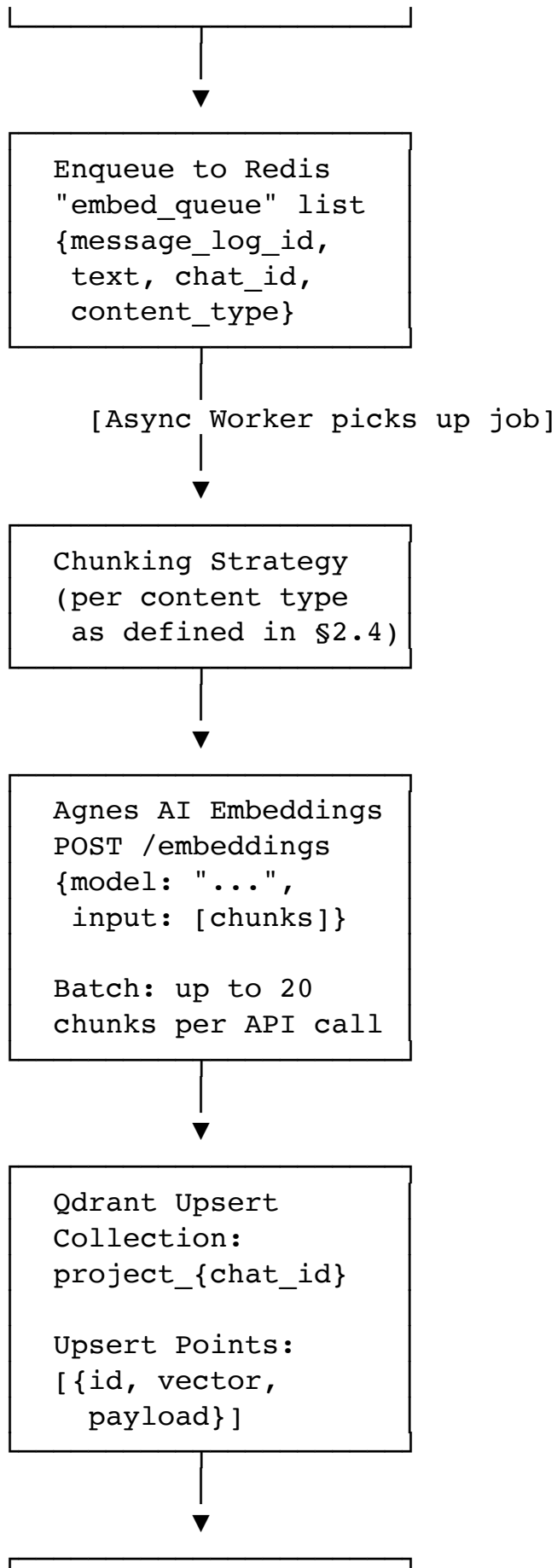
- Each web session opens one SSE connection to `/api/stream/updates`
- The backend SSE handler subscribes to all Redis channels for projects the student is a member of
- Heartbeat ping every 30 seconds to keep connection alive through proxies
- On reconnect, the client sends the last received `event_id`; backend replays missed events from PostgreSQL within a 5-minute window

5. RAG & Cache Cleansing Pipeline

5.1 Multi-Modal Ingestion Pipeline

Telegram Update (message/photo/document)





```

UPDATE message_logs
  is_vectorized=TRUE
  qdrant_point_ids=
    [generated UUIDs]

UPDATE projects
  qdrant_point_count
  += len(chunks)

```

5.2 RAG Query Execution Path

When Agnes calls `search_project_context`:

- 1 **Query Embedding:** The query string is embedded via Agnes AI's /`embeddings` endpoint
- 2 **Qdrant Search:** Search against `project_{chat_id}` collection only — no cross-collection queries possible at the API level
- 3 **Result Assembly:** Top-K points returned with their `payload.text_snippet` and metadata
- 4 **Context Injection:** Results are formatted as a structured block and inserted into Agnes's next message as `role: "tool"` content
- 5 **Deduplication:** Results are deduplicated by `message_log_id` to avoid the same source being cited twice from different chunks

5.3 Cache Cleansing — Operational Mechanics

Trigger mechanisms:

- **Manual (Bot):** A project lead sends `/clearcache` in the Telegram group. Bot confirms with a "Are you sure? Reply YES within 60 seconds." challenge.
- **Manual (Web):** Authenticated project lead clicks "Archive Project" in the dashboard, triggering a modal with explicit warning.
- **Automated:** A scheduled task (cron, daily at 02:00 SGT) checks for projects with `status = 'active'` and `qdrant_point_count > 100,000` and flags them for lead review (does not auto-clear without confirmation).

Cleansing Procedure (Transactional Order):

Step 1: AUTHORIZATION CHECK

- Verify requestor is a project member with role 'lead'
- For web: re-authenticate with password confirmation (elevated action)
- For bot: require YES confirmation in same group within 60s

Step 2: LOCK PROJECT (prevents new ingestion)

- `UPDATE projects SET status = 'clearing' WHERE id = {project_id}`

- Any incoming messages during this window are logged to message_logs
 - but NOT enqueued for embedding (worker checks project status)

Step 3: DELETE VECTOR STORE

- DELETE /collections/project_{chat_id} (Qdrant API)
- This is a single atomic API call – entire collection drops instantly
- On success: log to application audit log

Step 4: DELETE MINIO FILES (Optional, configurable)

- If "include_files=true" (user selected): delete entire bucket chat-{chat_id}/
- If "include_files=false": files are retained for manual review
- Default: retain files, clear only vectors

Step 5: SOFT-DELETE MESSAGE LOGS

- UPDATE message_logs
 - SET deleted_at = NOW(),
 - is_vectorized = FALSE,
 - qdrant_point_ids = '{}'
 - WHERE project_id = {project_id}
 - AND deleted_at IS NULL
- Preserves the message_logs rows for audit trail but marks them as cleared
- PostgreSQL Row-Level Security can hide these from normal queries

Step 6: RESET PROJECT STATUS

- UPDATE projects
 - SET status = 'archived', -- or 'active' if re-indexing is desired
 - qdrant_point_count = 0,
 - cleared_at = NOW()
 - WHERE id = {project_id}

Step 7: PRESERVE RELATIONAL INTEGRITY

- students, student_projects, tasks, deadlines, contribution_metrics:
 - UNTOUCHED. Core project data survives cache cleanse.
- Only the RAG corpus (vectors + raw logs) is removed.

Step 8: NOTIFY

- PUBLISH to Redis "project_updates:{project_id}" → type: "cache_cleared"
- Bot sends group notification: "Project context cleared by @username."
- Web dashboard updates project status badge to "Archived"

Rollback Guarantee: Steps 3–5 are not transactional at the database level (Qdrant and PostgreSQL are separate systems). To handle partial failures:

- Step 3 failure → abort, no data lost, status reset to 'active', alert raised
- Step 4 failure → Step 3 is already done (non-recoverable vectors). Log MinIO deletion failure to a `pending_deletions` table for a retry queue worker
- Step 5 failure → Qdrant is cleared but `message_logs` soft-delete failed. Worker retries Step 5 up to 3 times using the `project_id`. Idempotent operation.

6. Frontend UI/UX Technical Strategy

6.1 Next.js App Router Structure

```
app/
├── (auth)/                                # Route group – no shared
│   ├── layout with dashboard
│   │   ├── login/
│   │   │   └── page.tsx                # Server Component,
│   │   └── redirect if JWT valid
│   │       ├── register/
│   │       │   └── page.tsx
│   └── (dashboard)/                    # Route group – requires
│       ├── auth, has sidebar layout
│       │   ├── layout.tsx              # Root dashboard layout:
│       │   ├── Sidebar + SSE init
│       │   │   ├── page.tsx            # Overview: all projects,
│       │   │   └── upcoming deadlines
│       │   ├── projects/
│       │   │   ├── page.tsx            # Project list + "Link
│       │   │   └── New Project" CTA
│       │   │       ├── link/
│       │   │       │   └── page.tsx    # Project key input +
│       │   └── verification flow
│       │       └── [projectId]/
│       │           ├── page.tsx        # Per-project overview
│       │           ├── tasks/
│       │           │   └── page.tsx    # Kanban board (Pending/
│       └── In Progress/Done)
```

```

|   |   |   |— deadlines/
|   |   |   |   |— page.tsx      # Timeline view + Google
Calendar sync
|   |   |   |— members/
|   |   |   |   |— page.tsx      # Contribution radar
charts per member
|   |   |   |— context/
|   |   |   |   |— page.tsx      # RAG query UI — "Ask
about this project"
|   |— settings/
|   |   |— page.tsx              # Account settings
|   |   |— integrations/
|   |   |   |— page.tsx          # Google Calendar OAuth2
connect
|   |   |— telegram/
|   |   |   |— page.tsx          # Telegram username
verification status
|   |— notifications/
|   |   |— page.tsx
— api/
|   |— auth/
|   |   |— login/route.ts
|   |   |— register/route.ts
|   |   |— refresh/route.ts      # Token rotation endpoint
|   |— projects/
|   |   |— route.ts              # GET /api/projects
(list), POST /link
|   |   |— [id]/
|   |   |   |— tasks/route.ts
|   |   |   |— deadlines/route.ts
|   |   |   |— members/route.ts
|   |— stream/
|   |   |— updates/route.ts      # SSE endpoint
|   |— integrations/
|   |   |— google/
|   |   |   |— connect/route.ts  # OAuth2 redirect
|   |   |   |— callback/route.ts # OAuth2 callback
handler
|— middleware.ts                  # JWT validation on all
(dashboard) routes

```

6.2 Authentication Architecture (JWT)

Token Architecture:

Token	Storage	Lifetime	Purpose
Access JWT	httpOnly cookie	15 minutes	API authentication
Refresh Token	httpOnly cookie (Secure, SameSite=Strict)	7 days	Silent access token rotation

Why httpOnly cookies over localStorage?

- Immune to XSS token theft — JavaScript cannot read httpOnly cookies
- SameSite=Strict prevents CSRF on state-mutating requests
- Next.js middleware can validate the access JWT on the server before any page renders — no client-side redirect flash

JWT Payload:

```
{
  "sub": "uuid-of-student",
  "username": "john_doe",
  "telegram_verified": true,
  "project_ids": ["uuid1", "uuid2"],
  "iat": 1719302400,
  "exp": 1719303300
}
```

Middleware Logic (middleware.ts):

- 1 Extract `access_token` cookie
- 2 Verify JWT signature using `NEXTAUTH_SECRET`
- 3 Check expiry — if expired, attempt silent refresh via `/api/auth/refresh` using the `refresh_token` cookie
- 4 On successful refresh, set new `access_token` cookie and continue
- 5 On refresh failure, redirect to `/login` with `returnUrl` query param

6.3 State Management Strategy

Philosophy: Minimize client state. Prefer Server Components + server-side data fetching for initial page loads. Client state only for interactive real-time updates.

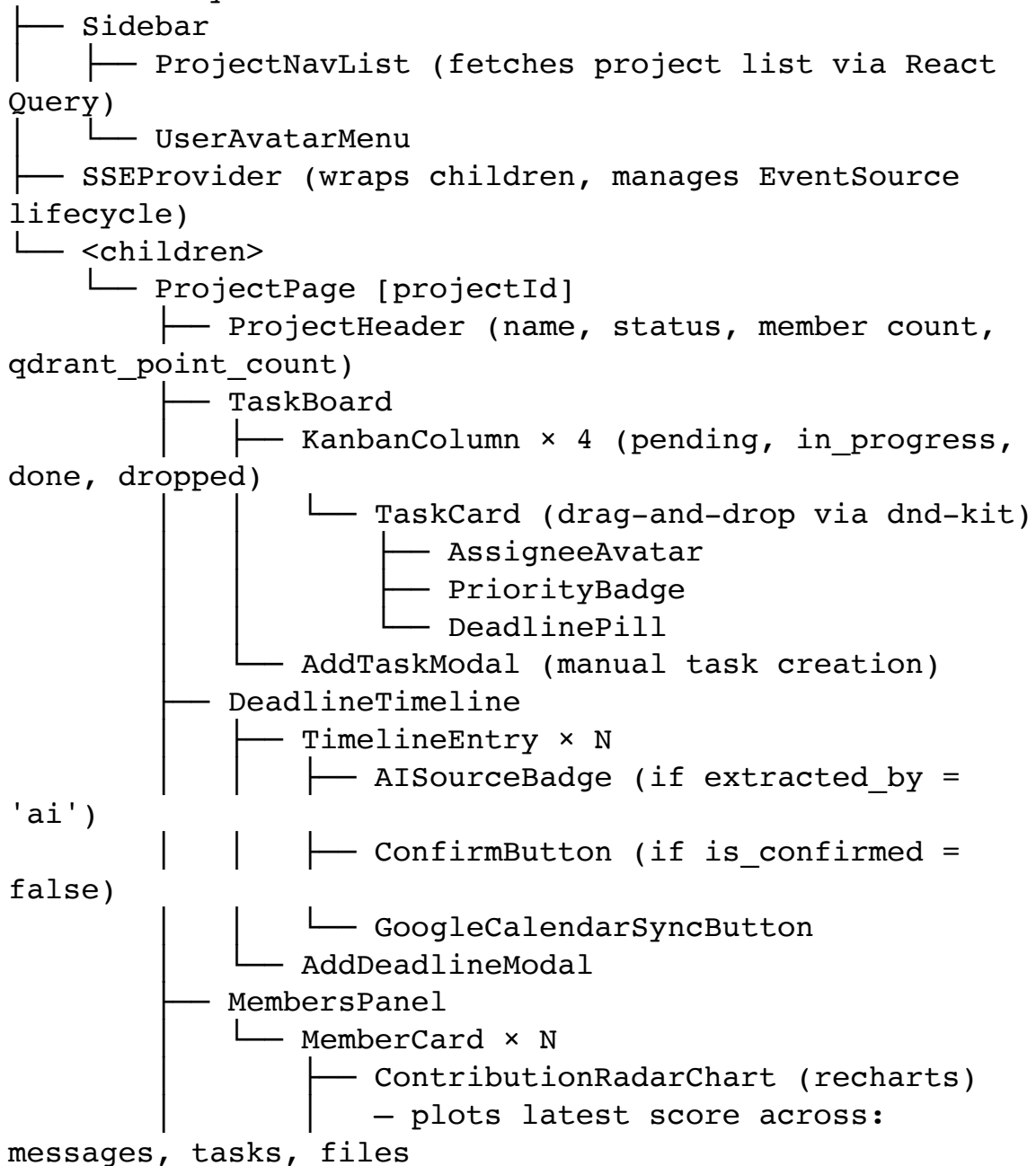
State Category	Solution	Rationale
Server data (tasks, deadlines)	TanStack Query (React Query)	Cache, background refresh, optimistic updates
Real-time SSE events	Custom React Context + EventSource	Simple, framework-native
Form state	React Hook Form + Zod	Performant, type-safe validation
UI state (modals, sidebars)	Zustand	Lightweight, no boilerplate
Auth state	React Query + cookie (no local storage)	JWT is in cookie; query fetches <code>/api/auth/me</code>

SSE Context Architecture:

- A single `SSEProvider` wraps the entire (dashboard) layout
- On mount, it opens one `EventSource` connection to `/api/stream/updates`
- Incoming events are dispatched to a central event bus (using a custom `EventEmitter` singleton)
- Individual components subscribe to specific event types (e.g., `TaskBoard` subscribes to `task_created`, `task_updated`)
- React Query's `queryClient.invalidateQueries()` is called on relevant events to trigger silent background refetches

6.4 Component Hierarchy (Key Sections)

DashboardLayout




```

      |           |
      |           |— ScoreHistory (sparkline)
      |           |— ScoreBreakdownAccordion (AI reasoning
text)  |
      |   |— ContextQueryPanel
      |   |   |— NaturalLanguageQueryInput
      |   |   |— QueryResultList
      |   |   |   |— ResultChunk (with source attribution)
      |   |   |— SuggestSummaryButton → calls /api/
      |   |       projects/[id]/summarize

```

6.5 Google Calendar Integration

Architecture: All OAuth2 exchanges happen **exclusively on the Next.js API routes** (server-side). The browser never sees the `client_secret` or `refresh_token`.

Flow:

Student clicks "Connect Google Calendar"



GET /api/integrations/google/connect

- Generates PKCE code_verifier + code_challenge (stored in signed session cookie)
- Generates state = HMAC(student_id + nonce) to prevent CSRF
- Redirects to Google OAuth2 consent screen with:
 - scope: calendar.events
 - access_type: offline
 - prompt: consent (forces refresh_token issuance)



▼ [User approves in Google]

GET /api/integrations/google/callback?code=...&state=...

- Verify state parameter (CSRF check)
- Exchange code for {access_token, refresh_token}
- ENCRYPT refresh_token with AES-256-GCM using server-side key
- UPDATE students SET google_calendar_refresh_token = encrypted_token
- Redirect to /settings/integrations with success flash

Sync Mechanics (Deadline → Calendar):

When a deadline is created or confirmed:

- 1 Backend fetches student's encrypted refresh token
- 2 Decrypts, exchanges for short-lived access token via Google Token Endpoint
- 3 Creates Google Calendar event via Google Calendar API v3

- 4 Stores `google_calendar_event_id` in `deadlines` table
- 5 On deadline update: `PATCH /calendar/v3/calendars/primary/events/{eventId}`
- 6 On deadline delete: `DELETE` the calendar event

Sync Mechanics (Calendar → Dashboard):

A background worker runs every 6 hours, scanning for Google Calendar changes using the `sync_token` pattern (Calendar's incremental sync API). Changes are reconciled with the `deadlines` table by matching `google_calendar_event_id`.

7. Infrastructure & Deployment Topology

7.1 Service Decomposition

Service	Technology	Scaling Model
Bot Process	Python + PTB, single async process	Vertical (single instance, webhook mode)
API Backend	FastAPI + Uvicorn + Gunicorn	Horizontal, stateless
Embed Worker	Python Celery worker	Horizontal, GPU-optional
Web Frontend	Next.js on Vercel or self-hosted	Edge-distributed or horizontal
PostgreSQL	Managed (Supabase/RDS) or self-hosted	Primary + Read Replica
Qdrant	Self-hosted Docker or Qdrant Cloud	Sharded by project volume
Redis	Managed (Upstash) or self-hosted	Single instance with AOF persistence
MinIO	Self-hosted	Erasure-coded cluster

7.2 Bot Deployment Model (Webhook vs. Polling)

Recommendation: Webhook mode for production.

- Telegram delivers updates via HTTP POST to `https://yourdomain.com/bot/webhook`
- Lower latency than polling, no wasted requests, mandatory for group chats at scale
- Requires a public HTTPS endpoint (use Caddy or Nginx as reverse proxy with Let's Encrypt)
- Single PTB `Application` instance handles all concurrent updates via `asyncio` event loop

7.3 Environment Variable Architecture

All secrets are stored in environment variables, never in code. Categories:

- `DATABASE_URL` — PostgreSQL connection string
- `QDRANT_URL` + `QDRANT_API_KEY` — Vector DB access

- `REDIS_URL` — Redis connection string
- `AGNES_AI_API_KEY` — Agnes AI credentials
- `AGNES_AI_BASE_URL` — `https://apihub.agnes-ai.com/v1`
- `TELEGRAM_BOT_TOKEN` — Bot authentication
- `TELEGRAM_WEBHOOK_SECRET` — Verifies webhook requests are from Telegram
- `PROJECT_KEY_HMAC_SECRET` — For generating `project_key` from `chat_id`
- `JWT_SECRET` + `JWT_REFRESH_SECRET` — Token signing keys (separate secrets)
- `GOOGLE_CLIENT_ID` + `GOOGLE_CLIENT_SECRET` — Calendar OAuth2
- `ENCRYPTION_KEY` — AES-256-GCM key for `refresh_token` at-rest encryption
- `MINIO_ACCESS_KEY` + `MINIO_SECRET_KEY` — File storage

8. Security Architecture

8.1 Threat Model Summary

Threat	Mitigation
Project key enumeration	HMAC-derived keys are 64-bit entropy; rate-limit <code>/api/projects/link</code> to 5 attempts/IP/hour
Unauthorized group access	Verification token requires physical presence in the Telegram group (sending <code>/verify</code>)
JWT theft	<code>httpOnly</code> cookies, HTTPS-only, <code>SameSite=Strict</code>
Cross-project RAG leakage	Per-collection Qdrant isolation; <code>chat_id</code> injected server-side, never from client
Prompt injection via chat	Agnes's <code>chat_id</code> is injected into system prompt by backend — cannot be overridden by user messages
Mass deletion via cache cleanse	Requires lead role + explicit YES confirmation; files retained by default
Google token compromise	Refresh tokens encrypted at rest (AES-256-GCM); access tokens never stored
File upload abuse	MIME type verification on MinIO upload; max file size 50MB enforced in bot handler

8.2 Data Privacy Boundaries

- **Telegram message content** is stored in `message_logs`. Privacy notice must be shown in the bot's welcome message.
- **Soft delete** on `message_logs.deleted_at` ensures data can be hard-deleted after a retention window (e.g., 90 days post-project-archive) via a cron job.
- **GDPR Right to Erasure:** `/api/students/me DELETE` endpoint hard-deletes the student row and sets `assigned_to_student_id = NULL` on their tasks. Message logs by that user are anonymized (username replaced with `[deleted]`)

rather than deleted (to preserve project history for other members).

Implementation Sequencing (Recommended Phases)

Phase 1 — Foundation (Weeks 1–2)

- PostgreSQL schema deployment (all tables, indexes, RLS policies)
- FastAPI skeleton with auth routes (register, login, JWT middleware)
- Qdrant collection management API
- Bot skeleton: webhook setup, basic command handlers (`/start`, `/help`)

Phase 2 — Core Bot Intelligence (Weeks 3–4)

- Multi-modal ingestion pipeline (text → image OCR → document parse)
- Embedding worker (Redis queue → Agnes AI → Qdrant upsert)
- Agnes AI integration with all 4 tool schemas
- `/ask`, `/deadlines`, `/tasks` bot commands

Phase 3 — Web Dashboard (Weeks 5–6)

- Next.js auth flows (register, login, JWT cookie management)
- Project linking flow (key input → token generation → Telegram verify)
- Task board (Kanban), deadlines timeline — read-only initially
- SSE infrastructure (Redis pub/sub → `/api/stream/updates` → client)

Phase 4 — Integration & Contribution (Weeks 7–8)

- Google Calendar OAuth2 flow + bidirectional sync
- Contribution metrics display (radar charts, score history)
- Cache cleansing UI (web) + bot command (`/clearcache`)
- RAG query panel on web (`/projects/[id]/context`)

Phase 5 — Hardening (Week 9–10)

- Rate limiting (slowapi on FastAPI, middleware on Next.js)
- Error handling: dead-letter queue for failed embedding jobs
- Load testing: simulate 10 active group chats with concurrent message ingestion
- Security audit: OWASP checklist, secret rotation procedure

This document represents the complete architectural specification for Student Claw v1.0. All implementation decisions should reference this document as the authoritative source of truth. Schema changes require a version increment and migration script.